

Competitive Programming

Section 31 Editorial: Sieve, Primes, and Factorization

Competitive Programming Course

2026-07-10

Table of contents

1	Editorial overview	3
2	Shared tool: an SPF table	4
3	A. Prime Counter	5
3.1	Restatement and constraints	5
3.2	Observations and walk-through	5
3.3	Pseudocode	5
3.4	Correctness proof	5
3.5	Complexity	5
3.6	Implementation notes and wrong answers	6
3.7	Full C++ solution	6
3.8	Full Python solution	6
4	B. Factor Signature	8
4.1	Restatement and constraints	8
4.2	Observations and walk-through	8
4.3	Pseudocode	8
4.4	Correctness proof	8
4.5	Complexity	9
4.6	Implementation notes and wrong answers	9
4.7	Full C++ solution	9
4.8	Full Python solution	10
5	C. Divisor Queries	11
5.1	Restatement and constraints	11
5.2	Observations and walk-through	11
5.3	Pseudocode	11
5.4	Correctness proof	11
5.5	Complexity	11
5.6	Implementation notes and wrong answers	12
5.7	Full C++ solution	12
5.8	Full Python solution	12
6	Testing and stress discussion	14
7	What to carry forward	15

1 Editorial overview

Problem	Preprocessing	Per query	Main trap
A. Prime Counter	sieve and prefix counts	$O(1)$	subtracting pref [1-1]
B. Factor Signature	SPF table	number of prime factors	x=1
C. Divisor Queries	SPF table	number of prime factors	multiply e+1

2 Shared tool: an SPF table

For B and C, `spf[x]` is the smallest prime dividing `x`; `spf[p]=p` remains true for a prime. The table uses indices `0..limit`, with `spf[1]=1` only as a sentinel. Its invariant is: after the sieve, `spf[x]` is a prime divisor of every `x>=2`.

```
spf[x] = x for x = 0..limit
for p from 2 while p*p <= limit:
    if spf[p] == p:
        for multiple from p*p to limit step p:
            if spf[multiple] == multiple:
                spf[multiple] = p
```

Building it costs $O(\text{limit} \log \log \text{limit})$ time and $O(\text{limit})$ memory. For each query, repeatedly divide out the current `spf`; each division removes one prime factor, so the local limit 10^6 makes this very small.

C++ note. Use `long long p` in `p*p` loop conditions. **Python note.** Read every query first so the table ends exactly at the maximum input value.

3 A. Prime Counter

3.1 Restatement and constraints

For up to 200,000 intervals inside $[1, n]$, where $n \leq 10^6$, count the primes in each inclusive interval. Scanning an interval per query can cost $O(nq)$, so the query must become constant time after one preprocessing pass. The boundary trap is $l=1$: there must be a valid `pref[0]`.

3.2 Observations and walk-through

Sieve once to obtain `is_prime`. Then let `pref[i]` count primes in $[1, i]$. For $n=10$, the primes are 2,3,5,7; therefore `pref[3]=2` and `pref[10]=4`. The answer for $[4, 10]$ is $4-2=2$, namely 5,7.

3.3 Pseudocode

```
mark 0 and 1 non-prime
for p from 2 while p*p <= n:
    if p is still prime:
        mark p*p, p*p+p, ... non-prime
pref[0] = 0
for i from 1 to n:
    pref[i] = pref[i-1] + is_prime[i]
for each (l, r): print pref[r] - pref[l-1]
```

3.4 Correctness proof

Every composite x has a prime divisor at most $\sqrt{x}$, so the sieve marks it while processing that divisor. A prime has no smaller divisor, so it is never marked. Thus `is_prime` is correct. By definition, `pref[r]` counts all primes through r and `pref[l-1]` counts exactly those before l ; their difference contains precisely the primes in $[l, r]$. $\square$

3.5 Complexity

```
Preprocessing:  $O(n \log \log n)$  time,  $O(n)$  memory
Each query:     $O(1)$  time
```

3.6 Implementation notes and wrong answers

In C++, `vector<bool>` is sufficient here; in Python, use a boolean list. Set both 0 and 1 to false before the loop. Do not use `pref[r]-pref[1]`: it silently drops a prime equal to 1. Do not start a fresh primality test for every endpoint or every query.

3.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<bool> is_prime(n + 1, true);
    is_prime[0] = false;
    if (n >= 1) is_prime[1] = false;
    for (long long p = 2; p * p <= n; ++p)
        if (is_prime[p])
            for (long long x = p * p; x <= n; x += p) is_prime[x] = false;
    vector<int> pref(n + 1);
    for (int i = 1; i <= n; ++i) pref[i] = pref[i - 1] + is_prime[i];
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << pref[r] - pref[l - 1] << '\n';
    }
}
```

3.8 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    is_prime = [True] * (n + 1)
    is_prime[0] = False
    if n >= 1:
        is_prime[1] = False
    for p in range(2, int(n**0.5) + 1):
```

```

        if is_prime[p]:
            for x in range(p * p, n + 1, p):
                is_prime[x] = False
    pref = [0] * (n + 1)
    for i in range(1, n + 1):
        pref[i] = pref[i - 1] + is_prime[i]
    out = []
    for i in range(2, 2 + 2 * q, 2):
        l, r = data[i], data[i + 1]
        out.append(str(pref[r] - pref[l - 1]))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

4 B. Factor Signature

4.1 Restatement and constraints

For each of up to 200,000 values $x \leq 10^6$, print: number of distinct prime factors, total factors with multiplicity, and largest prime factor. For $x=1$, print 0 0 1; this is a specified output convention because one has no prime factors.

4.2 Observations and walk-through

Use the shared SPF table. For $x=360$, the grouped divisions are:

current value	prime block removed	remaining	statistics
360	2^3	45	distinct 1, total 3, largest 2
45	3^2	5	distinct 2, total 5, largest 3
5	5^1	1	distinct 3, total 6, largest 5

The factors are encountered in increasing order because each is the smallest factor of the remaining value, so the last one is the largest.

4.3 Pseudocode

```
if x == 1: print 0 0 1
otherwise:
    distinct = total = 0; largest = 1
    while x > 1:
        p = spf[x]; exponent = 0
        while x is divisible by p:
            x /= p; exponent += 1
        distinct += 1; total += exponent; largest = p
    print distinct, total, largest
```

4.4 Correctness proof

At each iteration, $\text{spf}[x]$ is a prime divisor of the remaining x . Removing all copies records one and only one prime-power block. Unique factorization means these blocks are exactly all distinct prime factors, their exponents sum to the multiplicity count, and the final increasing block is the largest prime factor. The $x=1$ branch matches its empty factorization. $\square$

4.5 Complexity

```
Preprocessing:  $O(M \log \log M)$  time,  $O(M)$  memory,  $M = \max \text{ input}$   
Each query:  $O(\text{number of prime factors with multiplicity})$ 
```

4.6 Implementation notes and wrong answers

Keep the original value only if you need it later: the factor loop deliberately destroys x . Counting one distinct per division is wrong for $12=2^2*3$; increment it once per prime block. Do not print largest factor 0 for 1.

4.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

vector<int> build_spf(int n) {
    vector<int> spf(n + 1);
    iota(spf.begin(), spf.end(), 0);
    if (n >= 1) spf[1] = 1;
    for (long long p = 2; p * p <= n; ++p)
        if (spf[p] == p)
            for (long long x = p * p; x <= n; x += p)
                if (spf[x] == x) spf[x] = p;
    return spf;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    if (!(cin >> q)) return 0;
    vector<int> values(q);
    for (int &x : values) cin >> x;
    vector<int> spf = build_spf(*max_element(values.begin(), values.end()));
    for (int x : values) {
        if (x == 1) { cout << "0 0 1\n"; continue; }
        int distinct = 0, total = 0, largest = 1;
        while (x > 1) {
            int p = spf[x];
            ++distinct; largest = p;
            while (x % p == 0) { ++total; x /= p; }
        }
        cout << distinct << ' ' << total << ' ' << largest << '\n';
    }
}
```

4.8 Full Python solution

```
import sys

def build_spf(n: int) -> list[int]:
    spf = list(range(n + 1))
    if n >= 1:
        spf[1] = 1
    for p in range(2, int(n**0.5) + 1):
        if spf[p] == p:
            for x in range(p * p, n + 1, p):
                if spf[x] == x:
                    spf[x] = p
    return spf

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    values = data[1 : 1 + data[0]]
    spf = build_spf(max(values))
    out = []
    for x in values:
        if x == 1:
            out.append("0 0 1")
            continue
        distinct = total = 0
        largest = 1
        while x > 1:
            p = spf[x]
            distinct += 1
            largest = p
            while x % p == 0:
                total += 1
                x //= p
            out.append(f"{distinct} {total} {largest}")
    print("\n".join(out))

if __name__ == "__main__":
    main()
```

5 C. Divisor Queries

5.1 Restatement and constraints

For each of up to 200,000 values $x \leq 10^6$, print the number of positive divisors. Directly testing every candidate divisor up to x for each query is unnecessary; reuse the SPF factor blocks. The special case $x=1$ has answer one, not zero.

5.2 Observations and walk-through

For $x=72=2^3 \cdot 3^2$, a divisor chooses exponent 0,1,2,3 for prime 2 and 0,1,2 for prime 3. That is $4 \cdot 3 = 12$ choices. The choices are independent: selecting $2^1 \cdot 3^2$ cannot conflict with any other exponent selection.

5.3 Pseudocode

```
answer = 1
while x > 1:
    p = spf[x]; exponent = 0
    while x is divisible by p:
        x /= p; exponent += 1
    answer *= exponent + 1
print answer
```

5.4 Correctness proof

Unique factorization writes x as a product of prime powers. Every divisor is uniquely determined by choosing an exponent from $0 \dots e_i$ for every prime p_i , and every such selection divides x . There are e_i+1 choices independently, so the product computed by the algorithm is exactly the divisor count. $\square$

5.5 Complexity

```
Preprocessing:  $O(M \log \log M)$  time,  $O(M)$  memory
Each query:     $O(\text{number of prime factors with multiplicity})$ 
```

5.6 Implementation notes and wrong answers

Use an integer accumulator initialized to one. Adding exponents computes a different statistic. The local maximum divisor count fits in `int`, though `long long` is a harmless general habit. Ensure the `while x > 1` loop naturally leaves the answer as one for `x=1`.

5.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

vector<int> build_spf(int n) {
    vector<int> spf(n + 1);
    iota(spf.begin(), spf.end(), 0);
    if (n >= 1) spf[1] = 1;
    for (long long p = 2; p * p <= n; ++p)
        if (spf[p] == p)
            for (long long x = p * p; x <= n; x += p)
                if (spf[x] == x) spf[x] = p;
    return spf;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    if (!(cin >> q)) return 0;
    vector<int> values(q);
    for (int &x : values) cin >> x;
    vector<int> spf = build_spf(*max_element(values.begin(), values.end()));
    for (int x : values) {
        int answer = 1;
        while (x > 1) {
            int p = spf[x], exponent = 0;
            while (x % p == 0) { x /= p; ++exponent; }
            answer *= exponent + 1;
        }
        cout << answer << '\n';
    }
}
```

5.8 Full Python solution

```
import sys
```

```

def build_spf(n: int) -> list[int]:
    spf = list(range(n + 1))
    if n >= 1:
        spf[1] = 1
    for p in range(2, int(n**0.5) + 1):
        if spf[p] == p:
            for x in range(p * p, n + 1, p):
                if spf[x] == x:
                    spf[x] = p
    return spf

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    values = data[1 : 1 + data[0]]
    spf = build_spf(max(values))
    out = []
    for x in values:
        answer = 1
        while x > 1:
            p = spf[x]
            exponent = 0
            while x % p == 0:
                x //= p
                exponent += 1
            answer *= exponent + 1
        out.append(str(answer))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```

6 Testing and stress discussion

For A, compare against a slow loop that calls trial division on each number in a short interval. For B and C, compare factor-derived statistics with trial division on random values at most a few thousand. Essential fixed cases are:

```
A: [1,1], a singleton prime, and an interval with no primes  
B: 1, a large prime, 64, and 360  
C: 1, a prime, a prime power, and 210
```

Run the supplied generator with:

```
CP_TARGET=solution python3 sections/31_sieve_primes_factorization/check.py
```

7 What to carry forward

Use a sieve for properties of every value in a bounded range. Use SPF when the question needs the factorization itself. Once prime exponents are available, many apparently different number-theory statistics become one short loop.